

Intermediate Machine Learning: Assignment 1

Deadline

Assignment 1 is due Thursday, September 25 11:59 pm. Late work will not be accepted as per the course policies (see the syllabus on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged. Acknowledge any use of an AI system such as ChatGPT or Copilot.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Submission

Submit your assignment as a .pdf on Gradescope. You can access Gradescope through Canvas on the left-side of the class home page. The problems in each homework assignment are numbered. Note: When submitting on Gradescope, please select the correct pages of your pdf that correspond to each problem. This will allow graders to more easily find your complete solution to each problem.

To produce the .pdf, please do the following in order to preserve the cell structure of the notebook:

Go to "File" at the top-left of your Jupyter Notebook Under "Download as", select "HTML (.html)" After the .html has downloaded, open it and then select "File" and "Print" (note you will not actually be printing) From the print window, select the option to save as a .pdf

Topics

- Lasso
- Bias-variance decomposition
- Mercer kernels
- LOOCV for kernel smoothing and ridge regression

This assignment will also help to solidify your Python and Jupyter notebook skills.

Acknowledgement of Help

I used GPT 5 for help with these problems (mostly getting dimensions set up correctly, debugging, and streamlining my code), and I also benefitted greatly from discussing the problems with my classmate Jacey.

Problem 1: Roping variables with the lasso (15 points)

In this exercise, we'll employ the lasso regression technique to identify key predictor variables from the diabetes dataset. This dataset is used in the study by Efron, Hastie, Johnstone, and Tibshirani (2004, *Annals of Statistics*, "Least Angle Regression"). The primary goal is to predict a quantitative measure of diabetes progression one year after baseline, based on ten standardized physiological and biochemical measurements.

The *Diabetes* dataset in the `sklearn.datasets` package is a classic benchmark for regression. It contains 442 samples with 10 features that have been centered to mean zero but not scaled to unit variance. The predictors include age, sex, body mass index (BMI), blood pressure (BP), and six blood serum measures (s1–s6: cholesterol, LDL, HDL, cholesterol/HDL ratio, triglycerides, and blood sugar). The target is a quantitative measure of diabetes progression one year after baseline. You can find details at https://scikit-learn.org/stable/datasets/toy_dataset.html#diabetes-dataset.

Your tasks are as follows:

1. **Plotting Lasso Paths:** Generate a visualization of the lasso regularization paths.
2. **Identifying Key Predictors:** Determine which coefficients of (β) are non-zero.
3. **Estimating Coefficients:** Provide the best estimate for these non-zero coefficients.

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pickle
from sklearn.linear_model import Lasso
from sklearn.datasets import load_diabetes
%matplotlib inline
```

Just run the next cell to read in the data.

```
In [ ]: # Insert this code just above problem 1.1
X, y = load_diabetes(return_X_y=True)
n, p = X.shape
print("Number of rows: {}".format(n))
print("Number of columns: {}".format(p))
# Note that X, y = load_diabetes(return_X_y=True) has already been standardi
# You can explore the raw data with the following code:
X_raw, y_raw = load_diabetes(return_X_y=True, scaled=False)
print(np.allclose(y, y_raw))
print(np.allclose(X/np.std(X,axis=0), (X_raw - np.mean(X_raw, axis=0)) / np.
# The above two print statements should both return True.
```

Number of rows: 442
Number of columns: 10
True
True

1.1: Lasso regularization paths

Run the lasso and plot the regularization paths. You can use the `Lasso` class from the `sklearn.linear_model` package. Plot the parameter paths with the regularization level λ (`alpha` in the code) on the log-scale, as done in the lasso demo code from class. (As always, be sure to label your axes.)

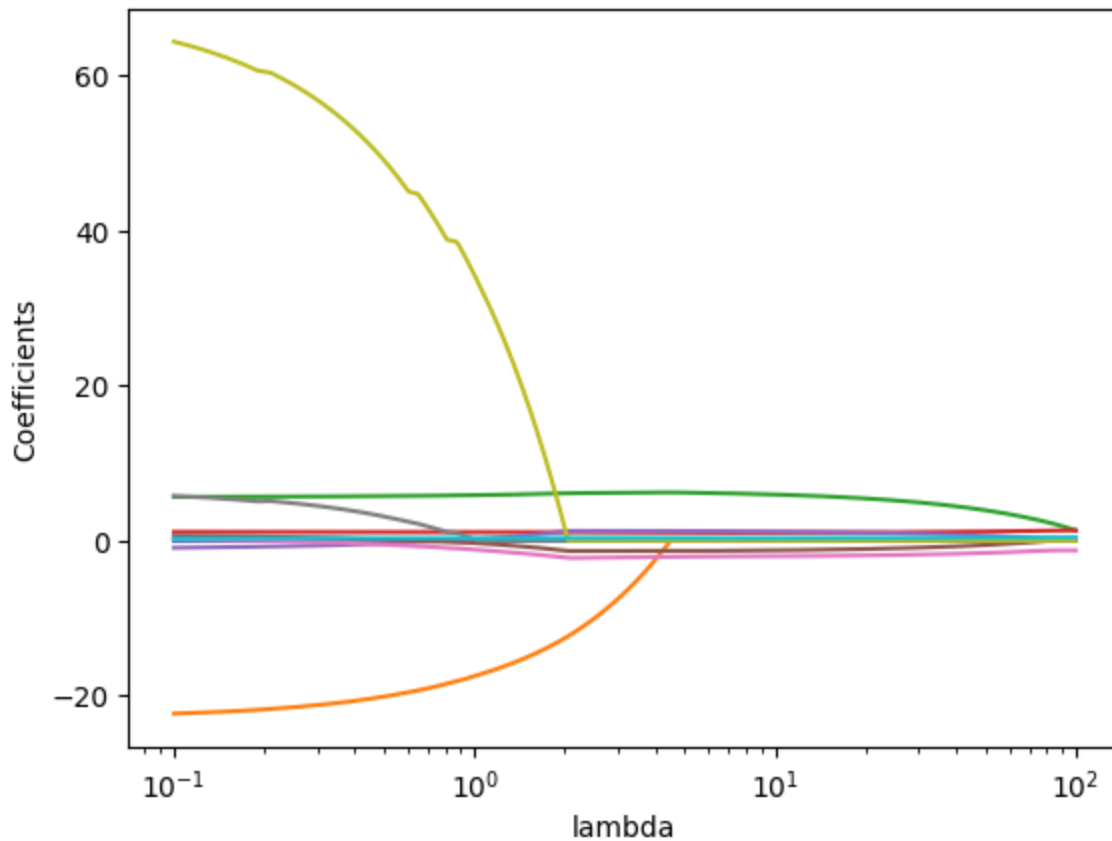
Show two plots, one where you run the lasso on the variables as given in the dataset, another where you standardize the variables to have mean zero and standard deviation one. Describe the differences in the regularization paths, and explain those differences.

When the predictors are standardized, what order do they appear in the lasso fits? That is, as λ decreases from infinity to zero, what is the sequence of variables that enter the model with nonzero coefficients? Explain why this ordering may (or may not) make sense.

```
In [ ]: # your code (and Markdown if needed) here
# Unstandardized coefficients
alphas = np.linspace(0.1, 100.0, 10000)
lasso = Lasso()
coefs = []

for a in alphas:
    lasso.set_params(alpha=a)
    lasso.fit(X_raw, y_raw)
    coefs.append(lasso.coef_)

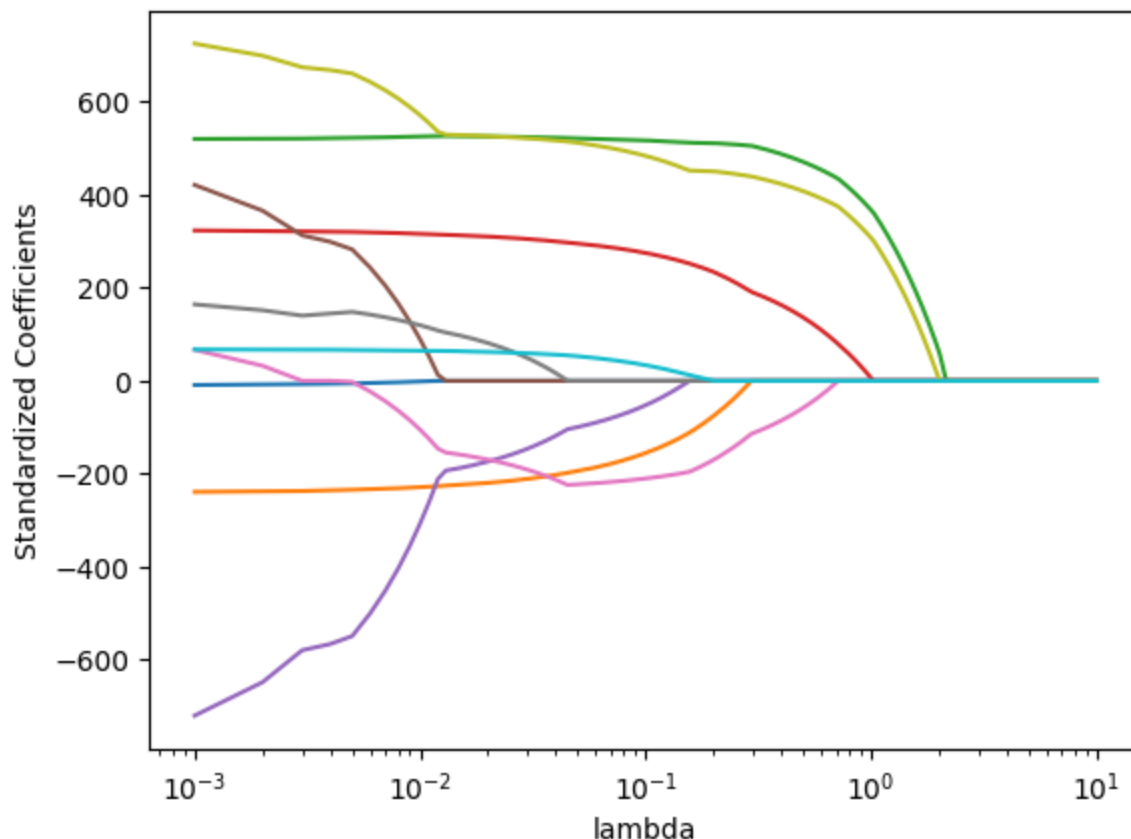
ax = plt.gca()
ax.plot(alphas, coefs)
ax.set_xscale('log')
plt.axis('tight')
plt.xlabel('lambda')
plt.ylabel('Coefficients')
plt.show()
```



```
In [ ]: # your code (and Markdown if needed) here
# Standardized coefficients
alphas = np.linspace(0.001, 10.0, 10000)
lasso = Lasso()
coefs = []

for a in alphas:
    lasso.set_params(alpha=a)
    lasso.fit(X, y)
    coefs.append(lasso.coef_)

ax = plt.gca()
ax.plot(alphas, coefs)
ax.set_xscale('log')
plt.axis('tight')
plt.xlabel('lambda')
plt.ylabel('Standardized Coefficients')
plt.show()
```



We can see that the scale of the variables matters a great deal here. In the unstandardized case, many of the coefficients start out close to zero, while a couple start out very far away; we worry that this may just be a scale issue. Indeed, many the variables that start relatively closer to zero don't seem to be zeroed out, while one of the large-coefficient variables shoots to zero once lambda reaches a certain point. The standardized coefficients look much better behaved.

In the standardized case, the coefficients that enter first are bmi and blood pressure, which makes sense to me (although I know very little about health/medicine) as they are known covariates with diabetes.

1.2: Select, estimate, and predict

The true model is linear, and only a subset $S \subset \{0, 1, \dots, 9\}$ of the 10 variables have non-zero coefficients β_j . In this problem you should make three estimates:

1. An estimate $\hat{S}$ of S
2. An estimate $\hat{\beta}_j$ for each $j \in \hat{S}$
3. An estimate of the predictive risk $\mathbb{E}(Y - X\hat{\beta})^2$

We are not specifying how you should construct these estimates. You should use your judgement, taste, and the tools provided from class. However, you must clearly explain and justify whatever approach that you use.

```

In [ ]: # your code (and Markdown if needed) here
# Below, I use leave-one-out-cross-validation (LOOCV) to determine the best
# So, we fit lasso at a given lambda, then use OLS to fit the data
# Then, compute LOOCV and try a different lambda

from sklearn.linear_model import Lasso, LinearRegression
from sklearn.model_selection import LeaveOneOut, cross_val_score
from sklearn.datasets import load_diabetes

# Setup
alphas = np.linspace(0.01,5,500)
lasso_coefs = []
important_variables = []
risk_estimates = np.inf

# Minimize risk using LOOCV
for a in alphas:
    lasso = Lasso(alpha = a)
    lasso.fit(X,y)
    nonzero_coefs_indices = np.nonzero(lasso.coef_)[0]
    if len(nonzero_coefs_indices) == 0:
        break
    important_vars_temp = X[:,nonzero_coefs_indices]
    ols = LinearRegression()
    scores = cross_val_score(ols, important_vars_temp, y, scoring = 'neg_mean_
    if np.mean(scores) < risk_estimates:
        risk_estimates = np.mean(scores)
        lasso_coefs = lasso.coef_[nonzero_coefs_indices]
        important_variables = nonzero_coefs_indices

diabetes = load_diabetes()
print('Variables that have non-zero coefficients (S hat): ',
      [diabetes.feature_names[i] for i in important_variables])
print('Coefficient estimates (beta hat) for the predictive variables: ',
      np.round(lasso_coefs,4))
print('The predictive risk is: ' , np.round(risk_estimates,4))

```

```

Variables that have non-zero coefficients (S hat):  ['bp']
Coefficient estimates (beta hat) for the predictive variables:  [56.5953]
The predictive risk is:  -3922.9885

```

The algorithm above performs the lasso for a given lambda value, then computes the estimated risk using LOOCV, then tries the lasso again for a different lambda value. In each iteration, I keep the run if it outperforms the best run I've done so far. In that way, I am basically doing a grid search over all the "possible" lambda values (defining the grid based on the figure from the standardized coefficients from part 1.1).

I find that, in the best run, I only keep one variable: BMI (body mass index).

Problem 2: Risky business (10 points)

In class (and in these notes) we sketched a proof that, when the regression function has two bounded derivatives, the bias and variance for kernel smoothing scale as

$$\text{bias}^2 = O(h^4)$$

$$\text{var} = O\left(\frac{1}{nh^p}\right).$$

Here h is the bandwidth parameter, n is the sample size, and p is the number of predictor variables. These expressions are asymptotic, meaning that they apply as n gets large and h gets small. In this problem your job is to reason about the implications of this bias-variance decomposition for prediction.

Note: For this problem, you may either enter your answers in Markdown using L^AT_EX, or you write them on paper and scan to insert as an image in the notebook; whichever you prefer.

2.1 Selecting the optimal bandwidth

Suppose that the bias and variance are such that

$$\text{bias}^2(\hat{m}(x)) \leq c_1 h^4$$

$$\text{var}(\hat{m}(x)) \leq c_2 \frac{1}{nh^p}.$$

for two constants c_1 and c_2 . Using these expressions and a little calculus, determine the optimal bandwidth h to minimize the risk function

$$R(h) = \mathbb{E}(\hat{m}(x) - m(x))^2.$$

Your answer should involve the constants c_1, c_2 , and n and p . Give a bound on the resulting risk using this bandwidth.

2.2 Bandwidth selection without tears

Now, going back to the expressions $\text{bias}^2 = O(h^4)$ and $\text{var} = O\left(\frac{1}{nh^p}\right)$, explain why the scaling of the optimal bandwidth (as a function of n and p), must satisfy $\text{bias}^2 \approx \text{var}$; that is, they must be of the same order as $h \rightarrow 0$. Then, without using any calculus, use this argument to determine the optimal scaling of the bandwidth and the fastest rate at which the risk $R(h) = \mathbb{E}(\hat{m}(x) - m(x))^2$ can approach zero as the sample size increases.

2.3 The cursed COD

Using the risk bound you derive above, make a plot that demonstrates the curse of dimensionality by plotting the sample size required to achieve a given level of risk. Specifically, let the target risk R vary between 0.1 and 0.5, and let the dimension p vary

between 1 and 20, and plot the sample size required to achieve that risk. Give a single plot that shows the collection of curves for each dimension.

2.1

$$\begin{aligned} R(h) &= \mathbb{E}(\hat{m}(x) - m(x))^2 \\ &= \text{bias}^2(\hat{m}(x)) + \text{var}(\hat{m}(x)) + \sigma^2 \\ &\leq c_1 h^4 + c_2 \frac{1}{n h^p} + \sigma^2 \end{aligned}$$

$$\hat{h} = \arg \min_h R(h)$$

$$\text{FOC: } 0 = \frac{\partial R(h)}{\partial h} = 4c_1 \hat{h}^3 - \frac{pc_2}{n} \hat{h}^{-p-1}$$

$$\Rightarrow \frac{pc_2}{n} \hat{h}^{-(p+1)} = 4c_1 \hat{h}^3$$

$$\Rightarrow \frac{pc_2}{4nc_1} = \hat{h}^{3+p+1}$$

$$\Rightarrow \hat{h} = \left(\frac{pc_2}{4nc_1} \right)^{\frac{1}{4+p}}$$

$$\text{SOC: } \frac{\partial^2 R(h)}{\partial h^2} = 12c_1 \hat{h}^2 + (p+1) \frac{pc_2}{n} \hat{h}^{-p-2} > 0$$

[so, indeed it is a minimum]

Upper bound on $R(h)$:

$$R(h) \leq c_1 \hat{h}^4 + \frac{c_2}{n \hat{h}^p} = c_1 \left(1 + \frac{4}{p}\right) \hat{h}^4 = c_1 \left(1 + \frac{4}{p}\right) \left(\frac{pc_2}{4nc_1}\right)^{4/(4+p)}$$

$$\text{So, } R(\hat{h}) = O(n^{-4/(4+p)})$$

2.2)

$$\text{bias}^2 = O(h^4), \text{var} = O\left(\frac{1}{nh^p}\right)$$

To minimize risk is equivalent to minimizing the sum of these two. Thus, at the minimum, both terms will be of the same order:

$$h^4 \asymp \frac{1}{nh^p}$$

$$\Rightarrow h^{4+p} \asymp \frac{1}{n}$$

$$\Rightarrow h \asymp n^{-\frac{1}{4+p}}$$

So, to find the rate at which risk decreases, we can substitute h back into either term:

$$R(h) \asymp h^4 = \left(n^{-\frac{1}{4+p}}\right)^4 = n^{-\frac{4}{4+p}}$$

which is the same result we showed in (2.1)

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt

# Risk rate:  $R \sim n^{-4/(p+4)}$ 
# Invert:  $n \sim R^{-(p+4)/4}$ 

# Define target risk values
R_vals = np.linspace(0.1, 0.5, 100)

# Define dimensions
```

```

p_vals = range(1, 21)

# Plot
plt.figure(figsize=(10, 6))

for p in p_vals:
    n_required = R_vals ** (-(p + 4) / 4)
    plt.plot(R_vals, n_required, label=f"p={p}")

# Log scale on y-axis
plt.yscale("log")

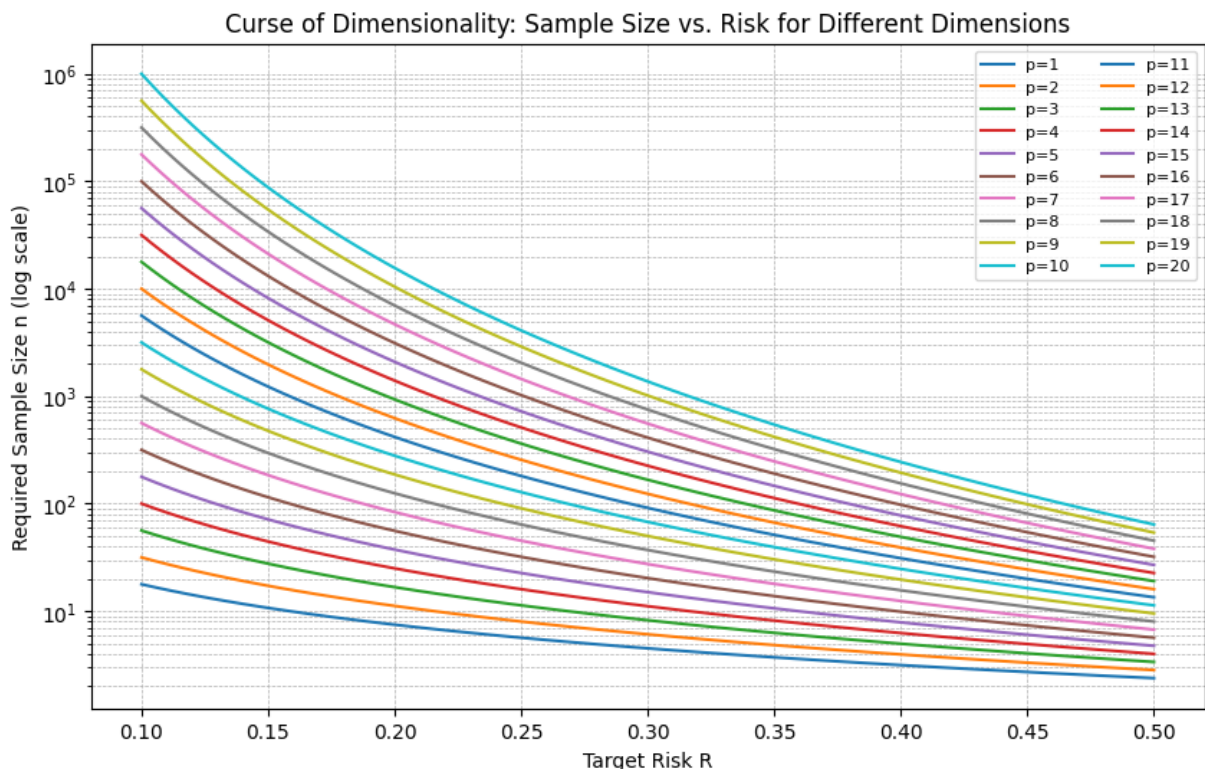
# Labels and title
plt.xlabel("Target Risk R")
plt.ylabel("Required Sample Size n (log scale)")
plt.title("Curse of Dimensionality: Sample Size vs. Risk for Different Dimen")

# Legend
plt.legend(ncol=2, fontsize=8)

# Grid
plt.grid(True, which="both", ls="--", lw=0.5)

plt.show()

```



Problem 3: A kernel of truth (15 points)

For problem you will implement nonparametric regression using Mercer kernels and penalization, in 1-dimension. This can be compared to regression using smoothing kernels.

As discussed in lecture, nonparametric regression with Mercer kernels is based on the infinite dimensional ridge regression

$$\hat{m} = \operatorname{argmin} \|Y - m\|^2 + \lambda \|m\|_K^2$$

By the representer theorem, this is equivalent to setting $\hat{m}(x) = \sum_{i=1}^n \hat{\alpha}_i K(X_i, x)$ and using the finite dimensional optimization

$$\hat{\alpha} = \operatorname{argmin} \|Y - \mathbb{K}\alpha\|^2 + \lambda \alpha^T \mathbb{K}\alpha$$

3.1 Solve

Derive a closed-form expression for the minimizer $\hat{\alpha}$. Show all of the steps in your derivation, and justify each step. (As above, you may either enter your answers in Markdown using $\text{\LaTeX}$, or insert an image of your handwritten solution.)

3.1

$$\hat{\alpha} = \arg\min_{\alpha} \{ \|Y - K\alpha\|^2 + \lambda \alpha' K \alpha \}$$

$$\text{FOC: } -2K'(Y - K\hat{\alpha}) + 2\lambda K\hat{\alpha} = 0 \leftarrow \text{calculus}$$

$$\Rightarrow -2K(Y - K\hat{\alpha}) + 2\lambda K\hat{\alpha} = 0 \leftarrow \text{linear algebra}$$

$$\Rightarrow -KY + K\hat{\alpha} + \lambda K\hat{\alpha} = 0$$

$$\Rightarrow K(K + \lambda I_n)\hat{\alpha} = KY$$

$$\Rightarrow \hat{\alpha} = (K + \lambda I_n)^{-1} K' KY$$

$$= (K + \lambda I_n)^{-1} Y$$

provided $(K + \lambda I_n)$ is nonsingular

3.2 Implement

Next you will use your solution above and implement Mercer kernel regression. We give some starter code.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
from IPython.display import clear_output
```

```
from time import sleep
```

The following cell defines some "helper functions" for this exercise. You don't need to change any of this code. (If you do want to make changes, just describe what you did and why.)

```
In [ ]: def plot_estimate(x, f, fhat, X, y, sigma, lambda, sleeptime=.01):
    clear_output(wait=True)
    plt.figure(figsize=(10,6))
    plt.plot(x, f, color='red', linewidth=2, label='true function')
    plt.plot(x, fhat, color='blue', linewidth=2, label='estimated function')
    plt.scatter(X, y, color='black', alpha=.5, label='random sample')
    plt.ylim(np.min(f)-4*sigma, np.max(f)+4*sigma)
    plt.legend(loc='upper left')
    plt.title('lambda: %.3g' % lambda)
    plt.xlabel('x')
    plt.ylabel('estimated m(x)')
    plt.show()
    sleep(sleeptime)

def true_fn(x):
    return 3*x**2

def run_simulation(kernel, lmbdas, show_bias_variance=True):
    min_x, max_x = -1, 1
    x = np.linspace(min_x, max_x, 100)
    f = true_fn(x)
    sigma = .25
    estimates = []
    trials = 500

    for lambda in lmbdas:
        estimates_lambda = []
        for i in np.arange(trials):
            X = np.sort(np.random.uniform(low=min_x, high=max_x, size=50))
            fX = true_fn(X)
            y = fX + sigma*np.random.normal(size=len(X))
            fhat = mercer_kernel_regress(kernel, X, y, x, lambda=lambda)
            if i % 50 == 0:
                plot_estimate(x, f, fhat, X, y, sigma, lambda)
            estimates_lambda.append(fhat)
        estimates.append(estimates_lambda)

    if show_bias_variance == False:
        return

    fhat = np.array(estimates)
    sq_bias = np.zeros(len(lmbdas))
    variance = np.zeros(len(lmbdas))

    for i in np.arange(len(lmbdas)):
        sq_bias[i] = np.mean((np.mean(fhat[i], axis=0) - f)**2)
        variance[i] = np.mean(np.var(fhat[i], axis=0))

    plt.figure(figsize=(10,6))
```

```
plt.plot(lmbdas, sq_bias, label='squared bias', linewidth=2)
plt.plot(lmbdas, variance, label='variance', linewidth=2)
plt.plot(lmbdas, sq_bias + variance, label='risk')
plt.legend()
plt.show()
```

Your job is to implement Mercer kernel regression and run it on two different kernel functions. The two kernels could simply be the Gaussian kernel with two different bandwidths, or you might experiment with other kernels.

The function `mercer_kernel_regress` takes a kernel, training data X and y , an array of values x to evaluate the function on, and a regularization parameter. You'll use your derivation above to determine the coefficients α . For some clues and suggestions on how to do the implementation, see our demo code for smoothing kernels. You need to do something very similar.

```
In [ ]: def mercer_kernel_regress(kernel, X, y, x, lambda):
        # your implementation here
        K = np.zeros((len(X), len(X)))
        for i in range(len(X)):
            for j in range(len(X)):
                K[i, j] = kernel(X[i], X[j])
        KX = np.zeros((len(x), len(X)))
        for i in range(len(x)):
            for j in range(len(X)):
                KX[i, j] = kernel(x[i], X[j])
        #alpha from 3.1 = (K + lambda*I_n)^(-1)*y
        alpha_hat = np.linalg.solve(K + lambda * np.identity(len(X)), y)
        fhat = KX @ alpha_hat
        return fhat

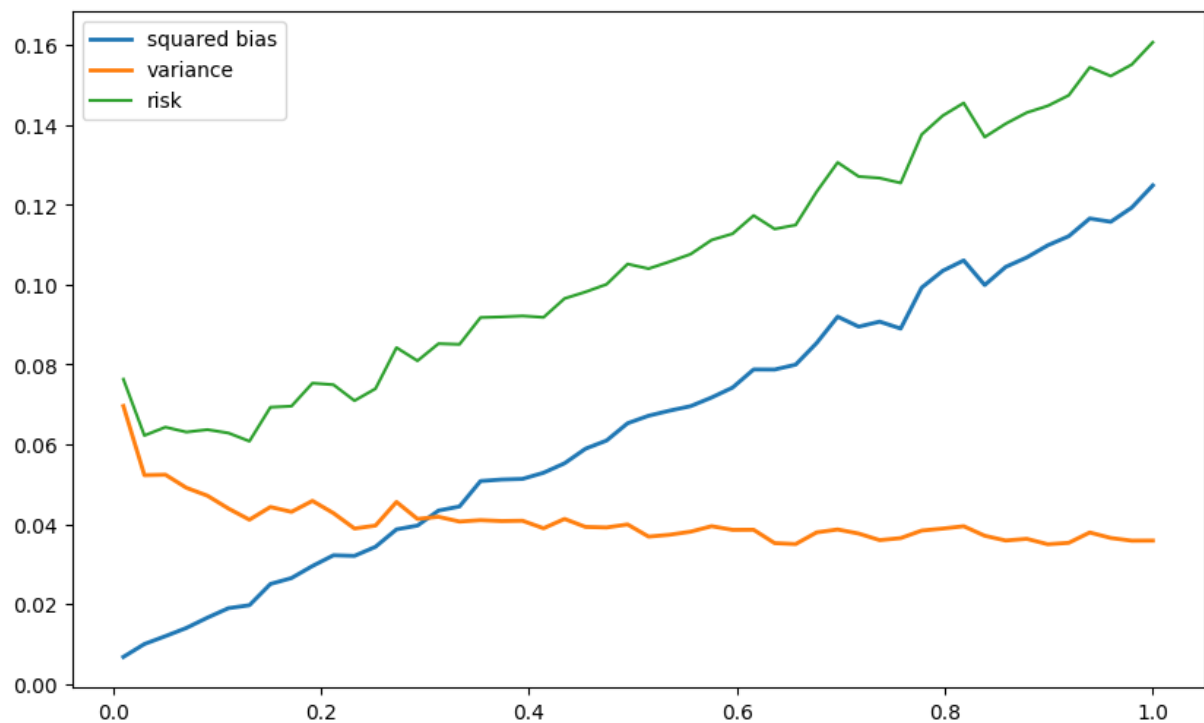
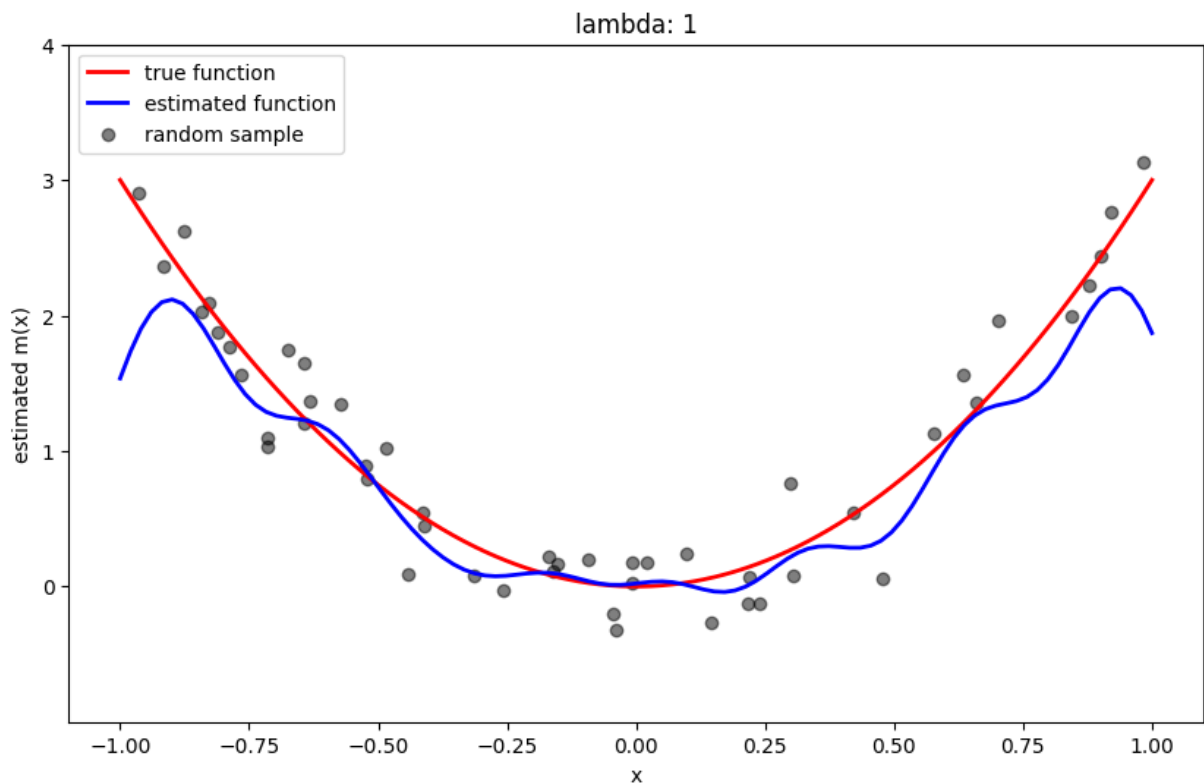
#I'm using Gaussians:
def kernel1(x,y):
    # your implementation here
    h = 0.1
    return(np.exp(-(x-y)**2/(2*h**2)))

def kernel2(x,y):
    # your implementation here
    h = 0.9
    return (np.exp(-(x-y)**2/(2*h**2)))
```

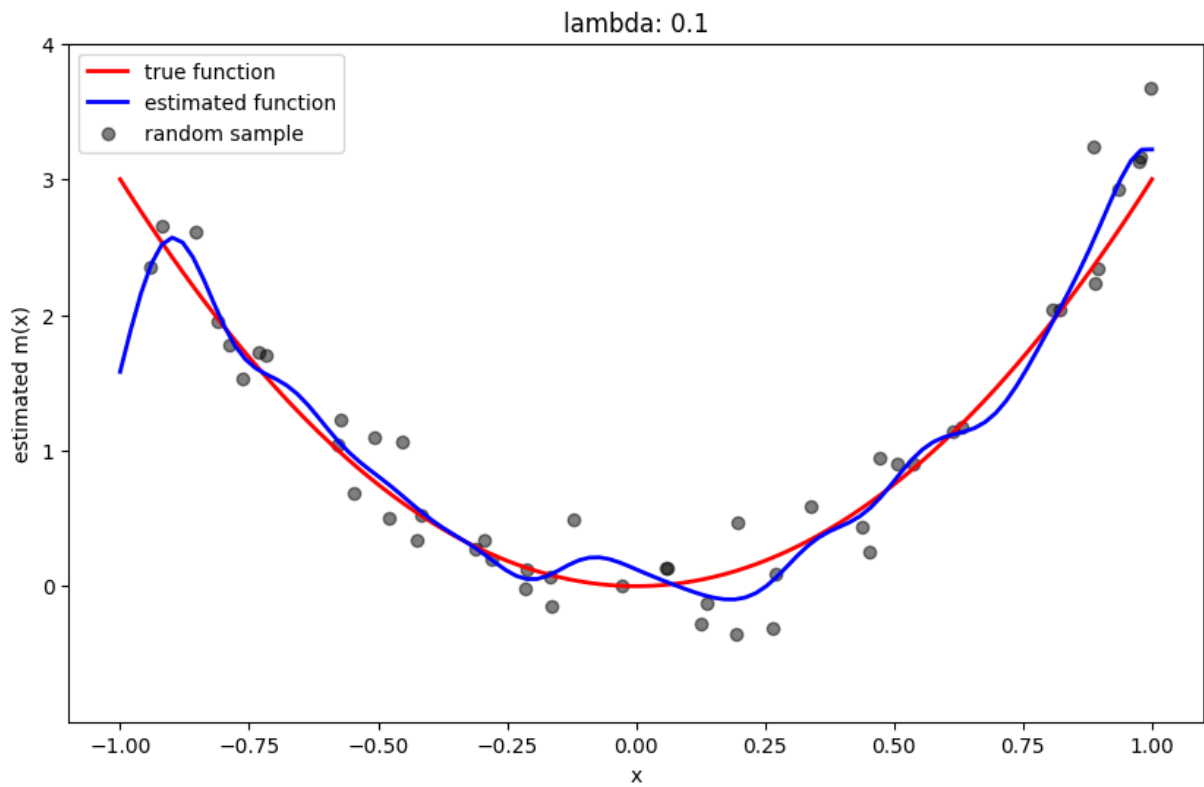
3.3 Run two simulations and select regularization parameters

Finally, using our starter code and your own implementation above, run two simulations, one using `kernel1` and the other using `kernel2`. After each simulation, select a regularization level from the bias-variance tradeoff, and then run a final simulation with that regularization level. In the following starter code, you only need to specify the sequence of regularization parameters.

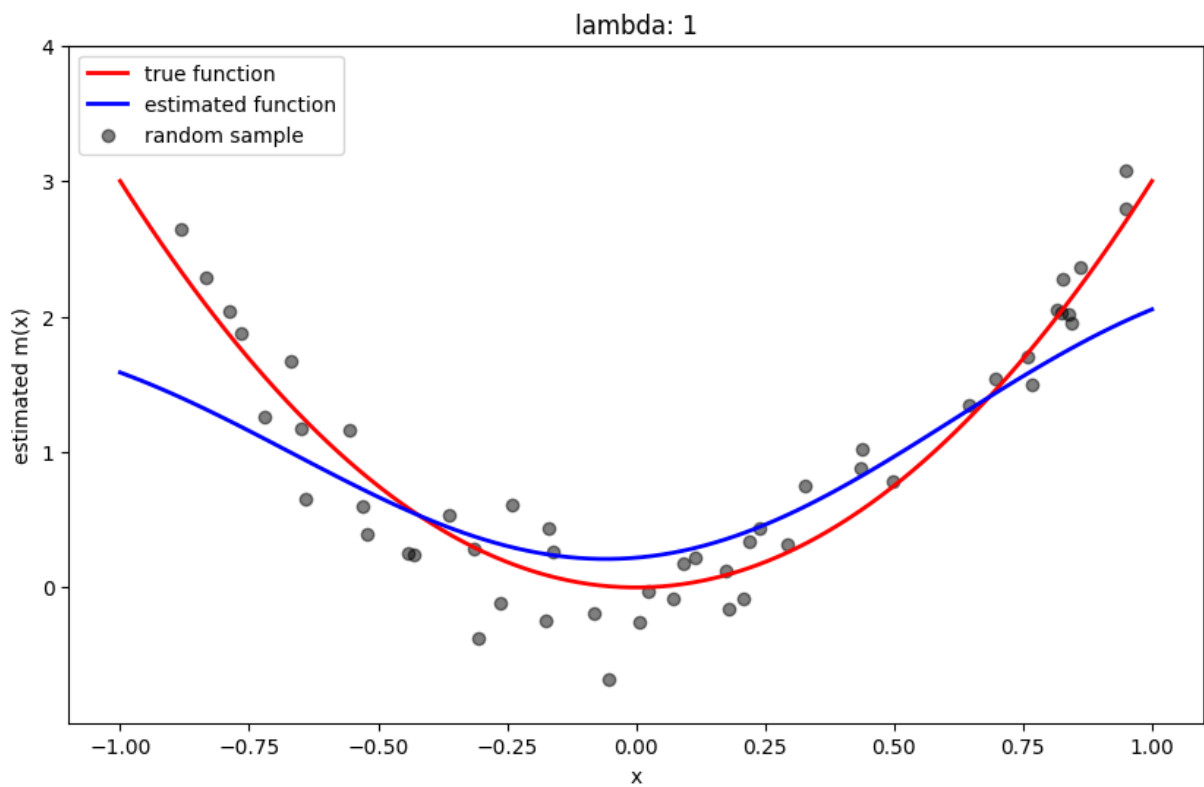
```
In [1]: lmbdas = np.linspace(0.01,1,50)
run_simulation(kernel1, lmbdas)
```

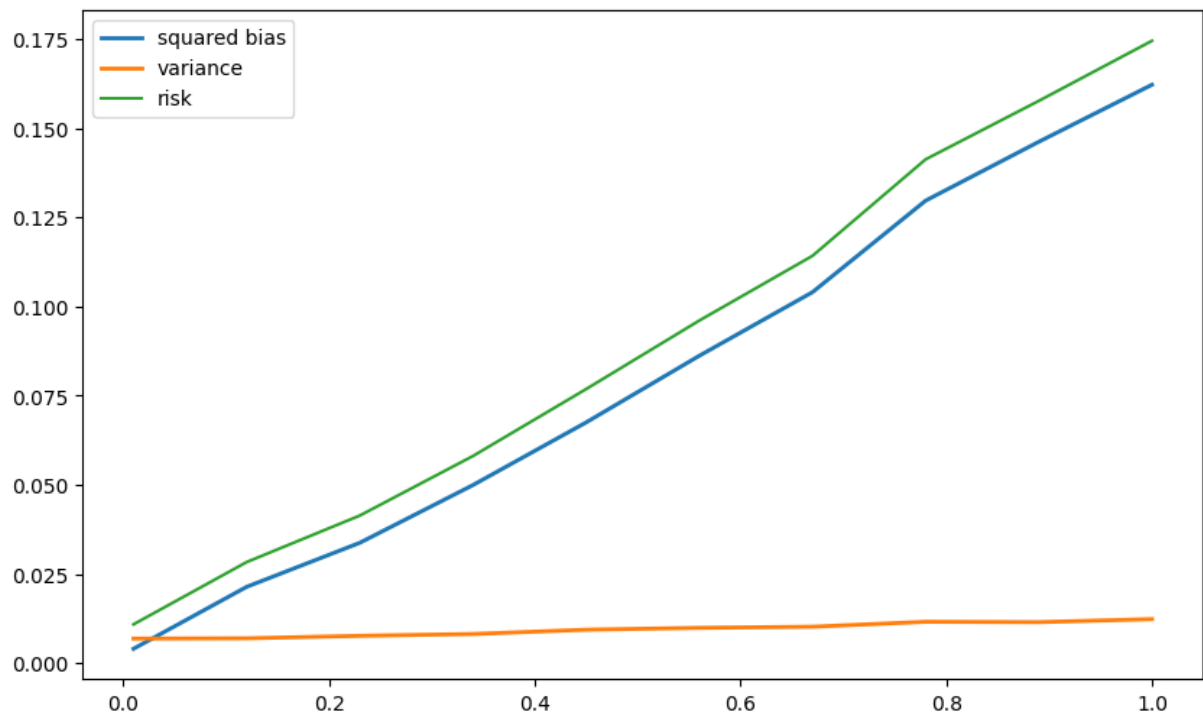


```
In [ ]: lambda_hat = 0.1
run_simulation(kernel1, [lambda_hat], show_bias_variance=False)
```

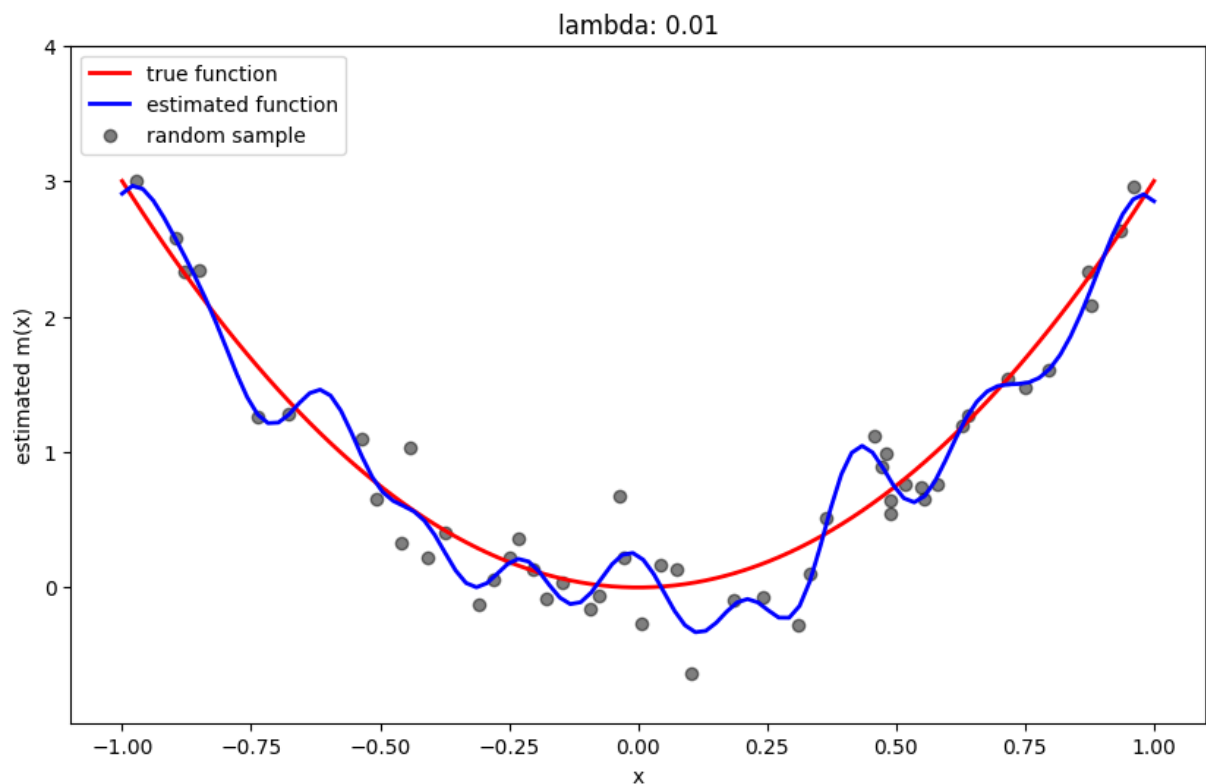


```
In [ ]: lmbdas = np.linspace(0.01, 1, 10)  
run_simulation(kernel2, lmbdas)
```





```
In [ ]: lambda_hat = 0.01
run_simulation(kernel1, [lambda_hat], show_bias_variance=False)
```



Problem 4: An algebraic simplification of LOOCV (15 points)

Leave-One-Out Cross Validation (LOOCV) is a specific type of K -fold cross validation where K equals the number of observations in the dataset. It works as follows for a training set with n observations:

1. A single observation is used as the validation set, and the remaining $n - 1$ observations serve as the training set.
2. A model is trained on the $n - 1$ observations and validated on the single left-out observation.
3. This process is repeated n times, each time leaving out a different observation as the validation set.
4. The LOOCV error is then the average error across all n trials.

LOOCV is particularly useful because:

- It utilizes almost all the data for training, so it's less prone to high variance compared to other validation schemes.
- Since each observation is tested exactly once, LOOCV provides a very thorough out-of-sample testing mechanism.

However, it can be computationally expensive because you have to fit the model n times. Luckily, for some models, there are algebraic simplifications available that make it computationally efficient. Expressing LOOCV in terms of the hat matrix allows for efficient computation of the LOOCV error without the need to refit the model for each left-out observation, making it a valuable tool for model evaluation.

Recall that the LOOCV error can be expressed as:

$$LOOCV = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_{-i})^2,$$

where $\hat{y}_{-i}$ represents the prediction for the i^{th} observation when it's left out from the training process. In the following questions, you will be deriving an alternative expression of the LOOCV error for both kernel and ridge regression, following the hints below.

1. LOOCV for kernel smoothing:

For kernels, we know that the LOOCV error can be equivalently written as the following form:

$$LOOCV = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - L_{ii}} \right)^2,$$

where $\hat{y}_i$ is the predicted value from the model fit on all data, and L_{ii} is the i^{th} diagonal element of the hat matrix L .

For kernel regression, we have

$$\hat{y} = Ly,$$

where

- $\hat{y}$ is the vector of predictions.
- y is the observed response values.
- L is the hat matrix and is defined by the kernel (for a given bandwidth). So, each diagonal element L_{ii} of the matrix L is defined as:

$$L_{ii} = \frac{K(x_i, x_i)}{\sum_{j=1}^n K(x_i, x_j)},$$

where

- K is the kernel function.
- x_i and x_j are the predictor values for observations i and j , respectively.

The diagonal elements L_{ii} give the "leverage" of each observation, which can be interpreted as the influence an observation has on its own prediction.

Derive this alternative expression of the LOOCV error for kernel regression. That's to say, for kernel regression, prove that

$$y_i - \hat{y}_{-i} = \frac{y_i - \hat{y}_i}{1 - L_{ii}}$$

4.1

$$\hat{y} = L y, \text{ where } L_{ii} = \frac{K(x_i, x_i)}{\sum_{j=1}^n K(x_i, x_j)}$$

$$\text{so } \hat{y}_i = \sum_{j=1}^n L_{ij} y_j \quad \leftarrow \begin{pmatrix} L_{11} & L_{12} & \dots & L_{1n} \\ \vdots & \ddots & & \vdots \\ L_{n1} & \dots & \dots & L_{nn} \end{pmatrix} \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$$

$$\text{Then, } \hat{y}_{-i} = \frac{\hat{y}_i - L_{ii} y_i}{1 - L_{ii}} \quad (\text{leaving out observation } i)$$

$$\begin{aligned} \text{So, } y_i - \hat{y}_{-i} &= y_i - \frac{\hat{y}_i - L_{ii} y_i}{1 - L_{ii}} \\ &= \frac{y_i - L_{ii} y_i - \hat{y}_i + L_{ii} y_i}{1 - L_{ii}} \end{aligned}$$

$$= \frac{y_i - \hat{y}_i}{1 - L_{ii}}$$

□

Problem 5: LASSO and Elastic Net

In class, we derived the solution for the one-dimensional LASSO using convexity and subgradients. We also know that coordinate descent can be used to solve LASSO in the high-dimensional case. Sometimes, we introduce a penalty that is a mixture of the ℓ_1 and ℓ_2 norms,

$$\lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2,$$

which is referred to as the **Elastic Net**. This problem demonstrates how the Elastic Net can be computed using the LASSO.

Let us first recall the one-dimensional LASSO objective:

$$f(\beta) = \frac{a}{2}\beta^2 - b\beta + \lambda|\beta|, \quad a > 0, \lambda \geq 0.$$

The minimizer is given by

$$\hat{\beta} = \frac{\text{sign}(b) (|b| - \lambda)_+}{a},$$

where $(x)_+ = \max\{x, 0\}$.

5.1 Solve the one-dimensional LASSO problem

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{2n} \sum_{i=1}^n (y_i - \beta x_i)^2 + \lambda |\beta| \right\},$$

where $\lambda \geq 0$ and at least one $x_i \neq 0$.

5.1

$$\min_{\beta} \frac{1}{2n} \sum_{i=1}^n (y_i - \beta x_i)^2 + \lambda |\beta|$$

$$\Leftrightarrow \min_{\beta} \frac{1}{2n} \left(\beta^2 \sum_{i=1}^n x_i^2 - 2\beta \sum_{i=1}^n x_i y_i + \sum_{i=1}^n y_i^2 \right) + \lambda |\beta|$$

$$\Leftrightarrow \min_{\beta} \underbrace{\frac{1}{2n} \left(\beta^2 \sum_{i=1}^n x_i^2 - 2\beta \sum_{i=1}^n x_i y_i \right)}_{\text{(drop constant)}} + \lambda |\beta| \quad (\text{drop constant})$$

$$= \frac{a}{2} \beta^2 - b\beta + \lambda |\beta|, \text{ where } a = n^{-1} \sum_{i=1}^n x_i^2 > 0, \text{ and } b = n^{-1} \sum_{i=1}^n x_i y_i$$

So we can use the result provided:

$$\hat{\beta} = \frac{\text{sign}(b)(|b| - \lambda)_+}{a} = \frac{1}{n^{-1} \sum_{i=1}^n x_i^2} \text{sign}\left(\frac{1}{n} \sum_{i=1}^n x_i y_i\right) \left(\left| n^{-1} \sum_{i=1}^n x_i y_i \right| - \lambda \right)_+$$

5.2 Solve the one-dimensional Elastic Net

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{2n} \sum_{i=1}^n (y_i - \beta x_i)^2 + \lambda_1 |\beta| + \lambda_2 \beta^2 \right\},$$

where $\lambda_1 \geq 0$, $\lambda_2 \geq 0$, and at least one $x_i \neq 0$.

Hint: Convert this problem into the form of 5.1. What happens if we observe a new data point $(\sqrt{2n\lambda_2}, 0)$?

5.2 | We follow the same steps as above, except this time we have an additional term in a :

$$\begin{aligned} \min_{\beta} \quad & \underbrace{\frac{1}{2n} \left(\beta^2 \sum_{i=1}^n x_i^2 - 2\beta \sum_{i=1}^n x_i y_i \right) + \lambda_2 \beta^2 + \lambda_1 |\beta|} \\ & = \frac{a}{2} \beta^2 - b\beta + \lambda_1 |\beta|, \text{ where} \\ & a = n^{-1} \sum_{i=1}^n x_i^2 + 2\lambda_2, \quad b = n^{-1} \sum_{i=1}^n x_i y_i \end{aligned}$$

$$\text{So, } \hat{\beta} = \frac{\text{sign}(b)(|b| - \lambda)_+}{a}$$

5.3 Now consider the case where we have data $\mathbf{Y} \in \mathbb{R}^n$ and $\mathbf{X} \in \mathbb{R}^{n \times p}$. We want to find an estimator

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{2n} \|\mathbf{Y} - \mathbf{X}\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2 \right\}.$$

Assume we have access to an oracle that can solve the LASSO. How can we use it to solve the Elastic Net?

Hint: Construct augmented variables $\mathbf{Y}^*$ and $\mathbf{X}^*$ (not necessarily the same shapes as $\mathbf{Y}$ and $\mathbf{X}$) and show that the objective is equivalent to

$$\frac{1}{2 \text{len}(\mathbf{Y}^*)} \|\mathbf{Y}^* - \mathbf{X}^* \beta\|_2^2 + \lambda_1 \|\beta\|_1.$$

5.3

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2n} \|\mathbf{Y} - \mathbf{X}\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2$$

Using the hint for 5.2, add the observation $(\sqrt{2n\lambda_2}, 0)$

$$\text{So let } \tilde{\mathbf{X}} = \begin{pmatrix} \mathbf{X} \\ \sqrt{2n\lambda_2} \mathbf{I}_p \end{pmatrix}, \tilde{\mathbf{Y}} = \begin{pmatrix} \mathbf{Y} \\ \mathbf{0}_p \end{pmatrix}$$

$$\text{Then, } \frac{1}{2n} \|\tilde{\mathbf{Y}} - \tilde{\mathbf{X}}\beta\|_2^2 = \frac{1}{2n} \|\mathbf{Y} - \mathbf{X}\beta\|_2^2 + \lambda_2 \|\beta\|_2^2$$

So now the original minimization becomes

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2n} \|\tilde{\mathbf{Y}} - \tilde{\mathbf{X}}\beta\|_2^2 + \lambda_1 \|\beta\|_1 \quad (\text{just Lasso again})$$

$$\text{Let } m = \text{len}(\mathbf{Y}^*), \mathbf{X}^* = \sqrt{\frac{n}{m}} \tilde{\mathbf{X}}, \mathbf{Y}^* = \sqrt{\frac{n}{m}} \tilde{\mathbf{Y}}$$

$$\text{Then } \frac{1}{2m} \|\mathbf{Y}^* - \mathbf{X}^* \beta\|_2^2 = \frac{1}{2n} \|\tilde{\mathbf{Y}} - \tilde{\mathbf{X}}\beta\|_2^2 = \frac{1}{2n} \|\mathbf{Y} - \mathbf{X}\beta\|_2^2 + \lambda_2 \|\beta\|_2^2$$

So, the original problem becomes

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2 \cdot \text{len}(\tilde{\mathbf{Y}})} \|\tilde{\mathbf{Y}} - \tilde{\mathbf{X}}\beta\|_2^2 + \lambda_1 \|\beta\|_1$$

5.4 Now revisit the diabetes dataset from the first problem. Apply the result you derived, together with the Lasso implementation from `sklearn.linear_model`, to solve the

elastic net. Specifically, we want to solve

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{2n} \|\mathbf{Y} - \mathbf{X}\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2 \right\},$$

where $\lambda_1 = \lambda_2 = 0.1$.

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pickle
from sklearn.linear_model import Lasso
from sklearn.linear_model import ElasticNet
%matplotlib inline
X, y = load_diabetes(return_X_y=True)
X = np.hstack([np.ones((X.shape[0], 1)), X])
n, p = X.shape

lambda1, lambda2 = 0.1, 0.1
# Follow X_tilde, Y_tilde setup from written answer above
I_pen = np.hstack([np.zeros((p-1, 1)), np.eye(p-1)])
X_tilde = np.vstack([X, np.sqrt(2 * n * lambda2) * I_pen])
y_tilde = np.concatenate([y, np.zeros(p-1)])
m = n + (p - 1)
c = np.sqrt(m / n)
X_star = c * X_tilde
y_star = c * y_tilde
model = Lasso(alpha=lambda1, fit_intercept=False)
model.fit(X_star, y_star)
your_coef = model.coef_

# You can use the following code to verify your answer
from sklearn.linear_model import ElasticNet
alpha = lambda1 + 2 * lambda2
l1_ratio = lambda1 / (lambda1 + 2 * lambda2)

# Fit ElasticNet with your X, y
model = ElasticNet(alpha=alpha,
                    l1_ratio=l1_ratio,
                    fit_intercept=False,
                    max_iter=10000,
                    tol=1e-6,
                    selection='cyclic')

model.fit(X, y)
elastic_coef = model.coef_
print(np.linalg.norm(your_coef - elastic_coef)/np.linalg.norm(elastic_coef))
```

0.1975710105991762

5.5 Which values of λ_1 and λ_2 are appropriate? A common approach is to use cross-validation. For simplicity, split the data once into training and validation sets, fit the model with different (λ_1, λ_2) , and compare the mean squared error (MSE) on the validation set.

```

In [ ]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, ra
scalerX, scalerY = StandardScaler(), StandardScaler()
X_train = scalerX.fit_transform(X_train)
X_test = scalerX.transform(X_test)
y_train = scalerY.fit_transform(y_train.reshape(-1,1)).ravel()
y_test = scalerY.transform(y_test.reshape(-1,1)).ravel()
grid = np.linspace(0.001, 1.0, 30)
params = []
mses = []
I_pen = np.hstack([np.zeros((p-1, 1)), np.eye(p-1)]) # shape: (p-1) x p
for lambda1 in grid:
    for lambda2 in grid:
        # fit a elastic net with lambda1 and lambda2 on (X_train, y_train)
        # compute the mse on (X_test, y_test)
        # you should use Lasso from sklearn. don't use ElasticNet.

        # Following the same steps as above again
        ntr = X_train.shape[0]

        # Augment design/response to encode the L2 penalty
        X_tilde = np.vstack([X_train, np.sqrt(2 * ntr * lambda2) * I_pen])
        y_tilde = np.concatenate([y_train, np.zeros(p-1)])

        # Scale so Lasso's (1/(2*len(y_star))) matches (1/(2*ntr)) +  $\lambda_2 ||\beta||$ 
        m = X_tilde.shape[0]
        c = np.sqrt(m / ntr)
        X_star = c * X_tilde
        y_star = c * y_tilde

        # Fit with Lasso
        model = Lasso(alpha=lambda1,
                      fit_intercept=False,
                      max_iter=10000,
                      tol=1e-6,
                      selection='cyclic')
        model.fit(X_star, y_star)

        # Evaluate on the test set
        y_pred = model.predict(X_test)
        mse = mean_squared_error(y_test, y_pred)

        params.append((lambda1, lambda2))
        mses.append(mse)
your_lambda1, your_lambda2 = params[np.argmin(mses)]

#You can verify your answer using the following code
params2 = []
mses2 = []

for lambda1 in grid:
    for lambda2 in grid:
        alpha = lambda1 + 2 * lambda2

```

```

l1_ratio = lambda1 / (lambda1 + 2 * lambda2)
model = ElasticNet(alpha=alpha,
                    l1_ratio=l1_ratio,
                    fit_intercept=False,
                    max_iter=10000,
                    tol=1e-6,
                    selection='cyclic')
model.fit(X_train, y_train)
mse = mean_squared_error(y_test, model.predict(X_test))
params2.append((lambda1, lambda2))
mses2.append(mse)
select_lambda1, select_lambda2 = params2[np.argmin(mses2)]

print(your_lambda1 == select_lambda1 and your_lambda2 == select_lambda2) #

```

True